

EXPERIMENT-16

Compare different classification algorithms and evaluate their performance.

Code:

```
import pandas as pd
from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

print("=" * 70)
print("    COMPARISON OF CLASSIFICATION ALGORITHMS")
print("=" * 70)

uploaded = files.upload()

filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)

print("\nDataset:")
print(data.head(10))
```

```

# Remove unnecessary columns
data = data.drop(
    columns=["id", "Unnamed: 32"],
    errors="ignore"
)

X = data.drop("diagnosis", axis=1)
y = data["diagnosis"]

encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Scaling for KNN and Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Logistic Regression":
        LogisticRegression(max_iter=5000),
    "Decision Tree":
        DecisionTreeClassifier(random_state=42),
    "KNN":
        KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes":
        GaussianNB()
}

print("\n" + "=" * 70)

```

```

print("MODEL PERFORMANCE")

print("=" * 70)

results = {}

for name, model in models.items():
    if name == "KNN" or name == "Logistic Regression":
        model.fit(X_train_scaled, y_train)
        prediction = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        prediction = model.predict(X_test)
    accuracy = accuracy_score(y_test, prediction) * 100
    results[name] = accuracy
    print("\nAlgorithm :", name)
    print("Accuracy :", round(accuracy, 2), "%")

print("\n" + "=" * 70)

print("FINAL COMPARISON")

print("=" * 70)

for name, accuracy in results.items():
    print(name, "->", round(accuracy, 2), "%")

best_model = max(results, key=results.get)
print("\nBest Performing Algorithm:")
print(best_model)
print("\nHighest Accuracy:")
print(round(results[best_model], 2), "%")

```

Sample Input:

```

...  =====
      COMPARISON OF CLASSIFICATION ALGORITHMS
      =====
      Choose Files data.csv
      data.csv(text/csv) - 125204 bytes, last modified: 8/10/2026 - 100% done
      Saving data.csv to data.csv

```

Sample Output:

```

Dataset:
...      id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0      842302         M        17.99         10.38         122.80        1001.0
1      842517         M        20.57         17.77         132.90        1326.0
2      84300903        M        19.69         21.25         130.00        1203.0
3      84348301        M        11.42         20.38          77.58         386.1
4      84358402        M        20.29         14.34         135.10        1297.0
5      843786         M        12.45         15.70          82.57         477.1
6      844359         M        18.25         19.98         119.60        1040.0
7      84458202        M        13.71         20.83          90.20         577.9
8      844981         M        13.00         21.82          87.50         519.8
9      84501001        M        12.46         24.04          83.97         475.9

      smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840         0.27760         0.30010         0.14710
1          0.08474         0.07864         0.08690         0.07017
2          0.10960         0.15990         0.19740         0.12790
3          0.14250         0.28390         0.24140         0.10520
4          0.10030         0.13280         0.19800         0.10430
5          0.12780         0.17000         0.15780         0.08089
6          0.09463         0.10900         0.11270         0.07400
7          0.11890         0.16450         0.09366         0.05985
8          0.12730         0.19320         0.18590         0.09353
9          0.11860         0.23960         0.22730         0.08543

... texture_worst  perimeter_worst  area_worst  smoothness_worst  \
0 ...          17.33          184.60        2019.0         0.1622
1 ...          23.41          158.80        1956.0         0.1238
2 ...          25.53          152.50        1709.0         0.1444
3 ...          26.50           98.87         567.7         0.2098
4 ...          16.67          152.20        1575.0         0.1374
5 ...          23.75          103.40         741.6         0.1791
6 ...          27.66          153.20        1606.0         0.1442
7 ...          28.14          110.60         897.0         0.1654
8 ...          30.73          106.20         739.3         0.1703
9 ...          40.68           97.65         711.4         0.1853

      compactness_worst  concavity_worst  concave points_worst  symmetry_worst  \
0          0.6656         0.7119         0.2654         0.4601
1          0.1866         0.2416         0.1860         0.2750
2          0.4245         0.4504         0.2430         0.3613
3          0.8663         0.6869         0.2575         0.6638
4          0.2050         0.4000         0.1625         0.2364
5          0.5249         0.5355         0.1741         0.3985
6          0.2576         0.3784         0.1932         0.3063
7          0.3682         0.2678         0.1556         0.3196
8          0.5401         0.5390         0.2060         0.4378
9          1.0580         1.1050         0.2210         0.4366

      fractal_dimension_worst  Unnamed: 32
0          0.11890         NaN
1          0.08902         NaN
2          0.08758         NaN
3          0.17300         NaN
4          0.07678         NaN
5          0.12440         NaN
6          0.08368         NaN
7          0.11510         NaN
8          0.10720         NaN
9          0.20750         NaN

```

[10 rows x 33 columns]

=====

MODEL PERFORMANCE

=====

Algorithm : Logistic Regression
Accuracy : 98.25 %

Algorithm : Decision Tree
Accuracy : 94.15 %

Algorithm : KNN
Accuracy : 95.91 %

Algorithm : Naive Bayes
Accuracy : 94.15 %

=====

FINAL COMPARISON

=====

Logistic Regression -> 98.25 %
Decision Tree -> 94.15 %
KNN -> 95.91 %
Naive Bayes -> 94.15 %

Best Performing Algorithm:
Logistic Regression

Highest Accuracy:
98.25 %